

Exerciții noi (setul 2) — vectori, stil LeetCode

Regulile jocului, ca până acum: datele se citesc din `data.txt` (prima linie `n`, a doua linie cele `n` numere). Pentru fiecare exercițiu scrii funcția cu semnătura dată în `functii-exte2.h` și o funcție `solutieN()` în `solutii-exte2.h` care citește, apelează și afișează. În `main.cpp` apelezi câte o singură `solutieN()`. **Nu modifica semnăturile.**

Exercițiile **2** și **3** au un parametru în plus pe **a treia linie** din `data.txt` (valoarea `x`) — acele `solutieN()` citesc fișierul direct.

Dificultate: ★ ușor ★★ mediu ★★★ provocare.

Nimic nou de învățat ca teorie: totul se rezolvă cu ce ai deja — **ștergere, inserare, frecvență, secvențe** și min/max. E doar antrenament pe aceleași unelte, pe probleme în stil LeetCode.

1. Suma maximă a unei secvențe

★★

LeetCode-ul original: Maximum Subarray (Kadane). [secvențe]

Se dă un vector care poate conține și numere negative. Găsește **suma maximă** a unei secvențe de elemente **consecutive** (cel puțin un element). Nu conta *care* e secvența, doar suma ei.

Exemplu — `data.txt`:

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Output: 6 (secvența 4 -1 2 1)

Verifică-te: vector cu toate elementele negative -3 -1 -2 → răspunsul e cel mai mare *element singur*, -1 (nu 0 — secvența trebuie să aibă măcar un element). Un singur element → chiar el.

Cum începi: cu **două bucle imbricate** — fixezi începutul secvenței cu `i`, aduni spre dreapta cu `j` și ții minte cea mai mare sumă văzută. Exact stilul de la recapitulare, răspuns complet corect.

Provocare (după ce merge): se poate și dintr-o **singură parcurgere**. Ții o „sumă curentă”; când ea scade sub 0, n-are rost s-o duci mai departe — o resetezi la 0 și pornești alta. E fratele mai deștept al lui `profitMaxim` de la setul 1.

```
int sumaMaxima(int v[], int n);
```

2. Șterge toate aparițiile unei valori

★★

LeetCode-ul original: Remove Element. [ștergere]

Se dă un vector și o valoare `x` (a treia linie din `data.txt`). **Șterge din vector toate aparițiile lui `x`, pe loc**, păstrând ordinea celorlalte elemente. Returnează noua lungime. Afișezi lungimea și primele elemente rămase.

Exemplu — `data.txt`:

```
6
3 2 2 3 4 3
3
```

Output: 3 și vectorul 2 2 4

Verifică-te: `x` nu apare deloc → vectorul rămâne neatins, lungime `n`; toate elementele egale cu `x` → lungime 0.

Cum o faci: exact tehnica de la `mutaZeroURI` (setul 1, problema 3) — un index `k` arată unde pui următorul element *care rămâne*; treci prin vector și copiezi la `v[k]` doar ce e diferit de `x`. Fără vector auxiliar.

```
int stergeValoarea(int v[], int n, int x);
```

3. Inserează într-un vector sortat

★★

LeetCode-ul original: *Search Insert Position*. [inserare]

Vectorul e **sortat crescător**. Se dă o valoare `x` (a treia linie). **Inserează** `x` pe poziția potrivită, astfel încât vectorul să rămână sortat. Elementele de după poziția aleasă se **mută cu una la dreapta**. Vectorul are acum `n+1` elemente — afișează-l.

Exemplu — `data.txt`:

```
5
1 3 5 7 9
6
```

Output: 1 3 5 6 7 9

Verifică-te: `x` mai mic decât toate → intră pe prima poziție; `x` mai mare decât toate → la coadă; `x` egal cu un element existent → lângă el (e corect oriunde între egale).

Cum o faci: găsești prima poziție `p` unde `v[p] >= x`. De acolo până la capăt, muți fiecare element cu o poziție la dreapta (**de la coadă spre p**, altfel te calci pe valori), apoi pui `v[p] = x`. Ai grijă ca `v` să aibă loc pentru elementul în plus.

```
int insereazaSortat(int v[], int n, int x);
```

4. Cel mai frecvent element

★★

LeetCode-ul original: *Majority / Mode*. [frecvență]

Găsește elementul care apare de **cele mai multe ori** în vector. Dacă mai multe elemente sunt la egalitate pe primul loc, afișează-l pe **cel mai mic** dintre ele. (Spre deosebire de „elementul majoritar” din setul 1, aici **nu** se garantează că apare de peste jumătate din ori.)

Exemplu — `data.txt`:

```
7
2 3 3 2 2 1 3
```

Output: 2 (2 și 3 apar amândouă de 3 ori — la egalitate afișezi cel mai mic, 2)

Verifică-te: toate elementele distincte 1 2 3 → fiecare o dată, afișezi cel mai mic, 1; un singur element → chiar el.

Cum o faci: pentru fiecare element, **numără de câte ori apare** în vector (o buclă interioară — fix frecvența de la recapitulare) și ține minte elementul cu frecvența cea mai mare; la egalitate, păstrează-l pe cel mai mic. Două bucle imbricate, în regulă.

```
int celMaiFrecvent(int v[], int n);
```

5. Cea mai lungă secvență strict crescătoare

★★

LeetCode-ul original: *Longest Continuous Increasing Subsequence*. [secvențe]

Găsește lungimea celei mai lungi **secvențe de elemente consecutive** în care fiecare e **strict mai mare** decât cel dinaintea lui.

Exemplu — data.txt:

```
9
1 2 1 2 3 4 1 2 5
```

Output: 4 (secvența 1 2 3 4)

Verifică-te: tot crescător 1 2 3 → n; tot descrescător 3 2 1 → 1 (fiecare element e o secvență de lungime 1); elemente egale 2 2 2 → 1 (egal nu e *strict* crescător, deci secvența se rupe).

Cum o faci: e fratele lui `maxUnuConsecutiv` (setul 1, problema 4). Ții o „lungime curentă”: dacă $v[i] > v[i-1]$ o crești, altfel o resetezi la 1; maxim reține cea mai mare valoare atinsă. O singură parcurgere.

```
int secventaCrescatoare(int v[], int n);
```

6. Produsul maxim a trei elemente

★★★★

LeetCode-ul original: *Maximum Product of Three Numbers*. [min/max]

Din vector, alege **trei elemente** (poziții distincte) al căror **produs** e cât mai mare. Afișează produsul maxim.

Exemplu — data.txt:

```
5
-10 -10 1 3 2
```

Output: 300 ($-10 \times -10 \times 3$, nu $3 \times 2 \times 1$)

Verifică-te: toate pozitive 1 2 3 4 → 24 (cele mai mari trei); dar când există **două negative mari**, produsul lor (pozitiv!) $\times$ cel mai mare pozitiv poate bate. Exact aici e capcana — $-10 \times -10 = 100$ întrece orice.

Cum începi: varianta sigură — **trei bucle imbricate** peste toate tripletele, ții produsul maxim. Corect, deși lent. Merge pentru n mic.

Provocare (după ce merge): nu-ți trebuie decât **cinci numere** din tot vectorul: cele mai mari trei valori și cele mai mici două. Răspunsul e *ori* produsul celor mai mari trei, *ori* al celor mai mici două (negative) înmulțit cu cea mai mare. Le găsești pe toate cinci dintr-o singură parcurgere — fix „al doilea maxim / primele minime” de la recapitulare, dus mai departe.

```
int produsMaximTrei(int v[], int n);
```

Ordinea recomandată

2 → 3 (ștergere și inserare — le știi deja, doar le exersezi) · 4 → 5 (frecvență și secvențe) · 1 (suma maximă, prima cu „resetează când nu mai merită”) · 6 (provocarea cu produsul — după ce restul merg fără ajutor).

Câte un commit per exercițiu, cu mesaj pe formatul feat: `exercitiul N - <nume>`. Nu trece la următorul până cel curent nu trece toate punctele de la „Verifică-te”.